TECHNICAL CASE STUDY

Message Notification Router

A personalized, multimodal triage system that decides which WhatsApp messages should interrupt you, which can wait, and which should never reach you at all.

Anit Kumar Maity
MCA, Pondicherry University
Dept. of Computer Science

HackerRank Orchestrate
24-hour hackathon · Aug 2026

110/110	**45**%	**3**	**3-tier**
messages routed to a valid decision — zero unhandled rows	of decisions resolved before a single LLM token was spent	modalities read natively — text, image (OCR), voice (ASR)	model fallback chain keeps the pipeline alive through a hard rate limit

The problem

Every chat app treats every message the same. WhatsApp can't afford to.

One inbox carries a family group, a school notice board, a co-worker's direct message, a verified business's promotion, and — wearing the same blue ticks as everyone else — an occasional scam. Notifying on all of it teaches people to ignore their phone. Muting too aggressively means a real emergency arrives in silence, unread, in a digest nobody opens in time.

personal	A direct message from a co-worker about a package mix-up needs a reply — same channel, completely different urgency than a group notice.
group	A muted family group should stay muted for a birthday forward, but not for a message that names you directly during a health emergency.
business	A payment reminder is legitimate from a business you've ordered from twice this month, and risky from a sender you've never transacted with.

Every message lands in one of three lanes

● `notify`
Important enough to interrupt the user right now.

● `digest`
Useful, but it can wait for a quieter moment.

● `mute`
Repetitive, unwanted, or unsafe for this specific user.

The judgment underneath is personal — same message, different sender history, different answer.

The pipeline

One message in, one validated decision out — five stages, always in this order.

1 Resolve content

Text passes through as-is. Images are read with `EasyOCR`; voice notes are transcribed with a small, int8-quantized `faster-whisper` model on CPU. Every message becomes plain text before anything downstream sees it.

2 Assemble context

Pulls the receiving user's quiet hours and recent open/reply/dismiss/report behavior, their role and mute state in the group, the sender business's verification and opt-out history, and current daily notification load.

3 Retrieve evidence

The resolved text is embedded with `sentence-transformers` (all-MiniLM-L6-v2) and matched against the user's history, boosted for same-thread and recent messages, paired with how the user reacted last time this sender wrote in.

4 Decide, in three tiers

A conservative rule layer resolves unambiguous cases without touching a model. The rest goes to Groq: Llama 3.3 70B, failing over to Llama 3.1 8B, then a local Ollama model, before any daily limit hits. A second rule layer then checks the model's answer against hard signals.

5 Validate

Every field is checked against the exact `action` / `message_type` contract. Cited evidence IDs are checked against what retrieval actually returned. A deterministic fallback guarantees a valid row even if every model tier fails.

WHY BATCH SIZE IS ONE

Grouping several messages into a single LLM call cuts token spend, but this pipeline defaults to one message per call. Personalization is per-user: asking a model to hold several users' contexts at once and keep judgments cleanly separated measurably hurt accuracy in testing. The batching path still exists in code for higher-volume runs — it's just off by default.

BUDGET-AWARE, NOT BUDGET-BLIND

Token cost is estimated before every call and tracked per model against Groq's daily token limit, with state persisted to disk so a second run later the same day remembers what the first one already spent. The pipeline switches models proactively, ahead of a 429, instead of discovering the wall mid-run.

Engineering decisions under a 24-hour clock

What a hackathon build has time for is exactly what makes the reliability choices matter more, not less.

01 · COST

Heuristics come first

Roughly **45 of every 100** messages hit a rule the pipeline is already confident about — a scam keyword, a heavily forwarded message from a low-engagement sender, a promotion from an opted-out business — and never reach a model at all. Token budget is reserved for the ambiguous majority, where a model's judgment is actually worth paying for.

02 · TRUST

A correction layer, not blind trust

15% of final decisions came from a second rule pass that overrides the model when it contradicts a hard signal — spam and scam are never allowed to `notify`, and a verified business with real order history is never blindly muted. The model proposes; ten explicit rules can veto.

03 · RELIABILITY

Degrade gracefully, don't crash

Groq's free-tier daily token limit is real and shared across a 24-hour build. A budget tracker estimates cost before every call, fails over to a smaller Groq model ahead of a 429, and drops to a local Ollama model as a last resort — so a long run never stalls partway through the dataset.

04 · INTEGRITY

No hallucinated citations

When the router cites a historical message as evidence, that ID is checked against what the retrieval step actually returned before it's allowed into the output. An invented ID is silently dropped rather than shipped — the citation contract is enforced in code, not assumed from the prompt.

> *"Muted for safety" and "notify — payment failed" are both one-line rules until you have to defend them against a real inbox. The rule layer exists because a prompt is a suggestion; a validated contract is a guarantee.*
>
> — design note, decision_engine.py

What the pipeline actually did

Behavior on the 110-message evaluation set, measured from the system's own output.

> HackerRank withholds the ground-truth labels for `messages.csv` until judging, so the numbers below describe the pipeline's own decision behavior — not accuracy against a hidden answer key. They're reported here for exactly that reason: no invented score, just what the system did.

ACTION DISTRIBUTION

notify	10 · 9%
digest	65 · 59%
mute	35 · 32%

DECISION PROVENANCE

pre-LLM heuristic	49 · 45%
LLM (unmodified)	44 · 40%
LLM + corrected	17 · 15%

INPUT MIX

text	87 · 79%
image (OCR)	15 · 14%
voice (ASR)	8 · 7%

Only 9% of traffic was routed to **notify**. That's a deliberate default, not a gap: the pipeline holds back unless a signal actually justifies an interruption, because a router that notifies on everything isn't a router.

STACK

`Python` `Groq · Llama 3.3 70B → Llama 3.1 8B` `Ollama (local fallback)` `sentence-transformers` `faster-whisper` `EasyOCR` `pandas` `tiktoken`

Built solo in 24 hours for HackerRank Orchestrate.
Code is a private hackathon submission — available on request.

Anit Kumar Maity
github.com/anitkumarmaity1-hash